VoidToken v5: Prospectively Frozen Evidence for KV-Cache Compression on a Real Language Model

Ivan Tyshchenko
Independent researcher
ORCID: 0009-0000-7935-6090
https://github.com/ALLPROTO/core-lm-benchmark

Abstract

We report an auditable, prospectively frozen evaluation of a serialized key-value (KV) cache codec on a pinned real language model. VoidToken v5 applies independent normalized Walsh–Hadamard transforms to the key and value halves of each layer cache, token-wise max-absolute scaling, symmetric quantization at 8 bits except 9 bits for layers 0 and 8, and canonical zlib compression. The complete wire containers, including framing, metadata, scales, and codes, are counted. The protocol fixes Qwen2.5-0.5B, WikiText-2 source windows, a canonical bfloat16 cache, wire-format byte accounting, statistical gates, and one-shot execution rules before frozen selection and the prospective holdout. On the registered 32-block holdout (4,096 teacher-forced predictions), complete containers reduced canonical bfloat16 prefill-cache storage from 150,601,728 to 73,346,513 bytes (2.05329×; 51.30% fewer bytes). Relative to canonical bfloat16-cache replay, the decoded cache produced $\Delta\text{NLL} = -6.09 \times 10^{-5}$ nat/token and 99.3896% top-1 agreement. The one-sided 95% block upper bound for ΔNLL was 0.000549, the block lower bound for agreement was 99.2472%, and the Wilson lower bound was 99.1543%; all seven prespecified gates passed. This is a bounded artifact result for one model revision, short teacher-forced windows, and one Apple-Silicon/MPS runtime. It is not a claim about full-model compression, free-running generation, latency, production serving, or state-of-the-art KV-cache quantization.

1 Introduction

Transformer inference stores attention keys and values so that earlier tokens need not be recomputed. The resulting KV cache grows with layer count, sequence length, and batch size, and is an important memory target for long-context inference. Recent work has therefore studied low-bit quantization, outlier handling, rotations, residual correction, pruning, and systems co-design [5, 2, 4, 3].

Most KV-cache papers ask how far precision can be reduced while retaining task quality or serving speed. This paper asks a narrower question: can one fixed codec configuration satisfy explicit quality and byte-accounting gates under a public-freeze workflow that rejects ordinary retries within the recorded runner and binds published outputs? The answer is positive for the registered experiment, but the scope is intentionally small.

The contribution is primarily experimental and procedural, not a claim of a new state of the art:

1. We specify a complete-container KV codec whose denominator, wire overhead, reconstruction path, and replay semantics are explicit, with executable checks enforcing them.
2. We separate adaptive development from a one-shot selection phase and a later prospective holdout, with public Git tags and durable attempt markers binding code, configuration, and evidence.
3. We publish block-level model outputs, exact artifact digests, recomputable confidence gates, schemas, and verifiers.
4. We report both favorable aggregate values and the limitations that prevent broader model, generation, device, or serving claims.

The final evidence tag is `voidtoken-v5-evidence-v1`, resolving to commit `531e4ab8`. The full commit and scientific digests are listed in Appendix A. The public repository contains the source, frozen records, and verification commands.

2 Related Work

KV-cache quantization. KIVI uses asymmetric quantization axes for keys and values and targets 2-bit storage [5]. KVQuant combines per-channel key quantization, pre-RoPE handling, non-uniform datatypes, and sparse outlier retention [2]. GEAR combines low-bit quantization with low-rank and sparse error correction [4]. These systems pursue compression and inference efficiency at substantially lower nominal precision than the 8/9-bit schedule evaluated here.

Rotations and serving-aware designs. QuaRot uses randomized Hadamard rotations to suppress outliers

across weights, activations, and KV caches [1]. KVLinC applies a Hadamard rotation plus learned linear correction for low-precision KV caches [9]. SAW-INT4 studies block-diagonal Hadamard rotations under paged-serving and fused-kernel constraints [3], while KVarN combines Hadamard rotation with variance normalization for autoregressive reasoning traces [7]. VoidToken v5 also uses a Hadamard transform, but it does not introduce rotation as a novel idea. Its distinctive target is an artifact-level, complete-container, prospective evaluation. Because no same-window implementations of these methods were run, this paper makes no comparative or state-of-the-art claim.

Model and corpus. The target is the base Qwen2.5-0.5B model from the Qwen2.5 family [8]. Text comes from WikiText-2, introduced with the WikiText corpus [6]. Model and corpus names alone are insufficient for reproduction, so the protocol pins repository revisions and file hashes.

3 Registered Evaluation Target

3.1 Model and corpus

The registered suite targets `Qwen/Qwen2.5-0.5B` at pinned revision `060db6499f32`. It has 24 layers, 14 attention heads, 2 KV heads, head dimension 64, and hidden size 896. The runner verifies the 988,097,824-byte weight file and its SHA-256 before loading. The full suite identifier, revision, and digest values appear in Appendix A.

The corpus is `Salesforce/wikitext`, `wikitext-2-raw-v1`, at pinned revision `b08601e04326`. Rows are retained in stored order, joined with exactly two newline characters, tokenized once without special tokens, and divided into non-overlapping 512-token blocks. There is no filtering, stripping, normalization, or index shifting.

3.2 Cache extraction

For every block, tokens 0–382 form the 383-token prefill. Each layer returns key and value tensors of shape $[1, 2, 383, 64]$. After removing the batch axis, the two KV heads are flattened token-major and key and value are concatenated:

$$X_\ell = [K_\ell \; V_\ell] \in \mathbb{R}^{383\times 256}, \qquad \ell = 0, \ldots, 23. \tag{1}$$

Native float32 cache values are rounded once through bfloat16 and restored to float32 for computation. This canonical cache is the common input to both the dense reference and codec. Its registered dense-byte total is

$$24 \cdot 383 \cdot 256 \cdot 2 = 4{,}706{,}304 \text{ bytes per block.} \tag{2}$$

The codec therefore compresses only a batch-1, 383-token prefill KV cache. It does not compress model weights or all process memory.

Before evaluating lossy reconstruction, two independently built canonical cache layouts must reproduce direct-cache continuation with zero maximum logit difference and identical top-1 decisions. This is the *structural replay* gate. It validates the extraction and reconstruction layout; it does not imply that the lossy codec is exact.

Continuation inputs are block tokens 383–510 and targets are tokens 384–511. Each block contributes 128 teacher-forced predictions conditioned on the supplied prefill cache. These continuation tokens are not recoded: continuation runs with `use_cache=False`, so no new KV state is appended and only the fixed prefill cache differs between replay paths.

4 VoidToken v5 Codec

For each row of X_ℓ, the key and value halves form two independent 128-wide groups. Let H_{128} be the orthonormal Walsh–Hadamard matrix. The forward transform is

$$Y_{\ell,g} = X_{\ell,g} H_{128}, \qquad g \in \{K, V\}. \tag{3}$$

No pseudo-random sign rotation is used. For every token row and K/V group, a max-absolute quantization step is rounded to little-endian float16. Layers 0 and 8 use signed 9-bit quantization; the remaining 22 layers use signed 8-bit quantization. Integer codes use a fixed zigzag mapping. Scale bytes and packed codes are compressed separately with canonical zlib level 9.

Each layer is serialized as

$$\texttt{"VTL5"} \mid \texttt{uint32-le}(m) \mid \texttt{canonical JSON}_m \mid \texttt{payload}. \tag{4}$$

All 24 complete containers count toward the compressed-byte total, including magic bytes, length fields, metadata, scale streams, and code streams. Model replay receives only arrays reconstructed from a fresh strict parse of these serialized bytes. The parser is a separate byte entry path, but it shares the canonical payload decoder with the encoder self-check and is not claimed as an independent implementation.

Figure 1 summarizes the counted wire path and the two replay branches.

5 Prospective Evidence Protocol

5.1 Data partition

The data roles are fixed and disjoint:

- validation blocks 0–31: adaptive codec development;
- validation blocks 32–63: one-shot frozen selection;
- test blocks 384–415: prospective holdout;
- test blocks 416–447: unscored reserve, inaccessible to the runner.

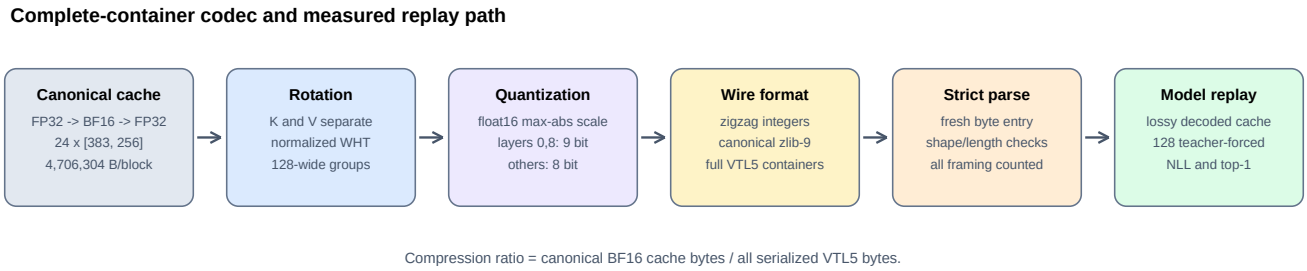

Figure 1: VoidToken v5 codec and replay path. The baseline is a canonical bfloat16-rounded prefill cache, and the ratio counts complete serialized containers. The decoded model replay is lossy.

Development records are disclosed but do not count toward the prospective verdict. The reserve is unscored, not secret or unread: corpus construction tokenizes the complete public parquet before slicing registered windows. Moreover, an earlier exploratory v1 pilot had already evaluated disjoint test blocks 8–15. These facts limit the strength of any "unseen data" language.

5.2 Public freezes and one-shot locks

The fixed implementation, codec, thresholds, model, corpus, and selection indices were published under a lightweight protocol tag at commit `46753887`. The one-shot selection then ran at that commit. After its PASS result and exact attempt marker were committed, a lightweight pretest tag was published at commit `34fbd055`. Only then could the holdout runner resolve the test split. Full tag and commit values appear in Appendix A.

Each frozen phase creates a fixed attempt marker with exclusive creation and durable file and directory synchronization after environment and public-tag checks but before resolving its registered split. An existing marker consumes the phase. A crash after marker creation is terminal `CONSUMED_INCOMPLETE`; a scientific FAIL is published unchanged. There is no ordinary retry path and no CLI option for blocks, codec parameters, thresholds, revisions, or device.

These controls are strong process evidence, not cryptographic proof that no person deleted a local file or ran altered private code. Git tags are also mutable by repository administrators. The protocol therefore claims a recorded one-shot execution corroborated by public history and artifact binding, not an append-only trust system.

Figure 2 shows where adaptive work ends and the frozen selection and holdout phases begin.

6 Metrics and Fixed Gates

For each prediction, both paths use the same target and differ only in the prefill cache. Define

$$\Delta\mathrm{NLL} = \mathrm{NLL}_{\mathrm{decoded}} - \mathrm{NLL}_{\mathrm{canonical\ BF16}}. \tag{5}$$

Top-1 agreement is agreement between decoded-cache and canonical-cache next token decisions; it is not predictive accuracy against a benchmark label. Writing B_{BF16} for the canonical dense-byte total in Eq. 2 and $B_{\mathrm{container}}$ for the actual bytes of all 24 complete containers, the reported compression ratio is

$$\rho = \frac{B_{\mathrm{BF16}}}{B_{\mathrm{container}}}. \tag{6}$$

For $n = 32$ equal-sized blocks, let $\bar{d}$ and s_d be the sample mean and standard deviation of block ΔNLL. The registered one-sided upper bound is

$$U_d = \bar{d} + t_{0.95,31}\frac{s_d}{\sqrt{32}}, \qquad t_{0.95,31} = 1.6955187825. \tag{7}$$

The analogous lower bound for block top-1 rates is $L_a = \bar{a} - t_{0.95,31}s_a/\sqrt{32}$. A one-sided Wilson score lower bound over all 4,096 token decisions uses $z = 1.644853627$ [10, 11]. Adjacent teacher-forced decisions are correlated, so Wilson is a frozen workflow gate rather than independent Bernoulli population inference. The block-level Student's t bound is the registered block-aggregated backstop, but 32 contiguous blocks still do not define a broad population.

Selection and holdout must each satisfy all seven gates without rounding:

1. complete-container ratio ≥ 2.0;
2. aggregate ΔNLL ≤ 0.01 nat/token;
3. $U_d \leq 0.01$ nat/token;
4. aggregate top-1 agreement ≥ 0.99;
5. blockwise $L_a \geq 0.99$;
6. Wilson lower bound ≥ 0.99;
7. exact structural replay on all blocks.

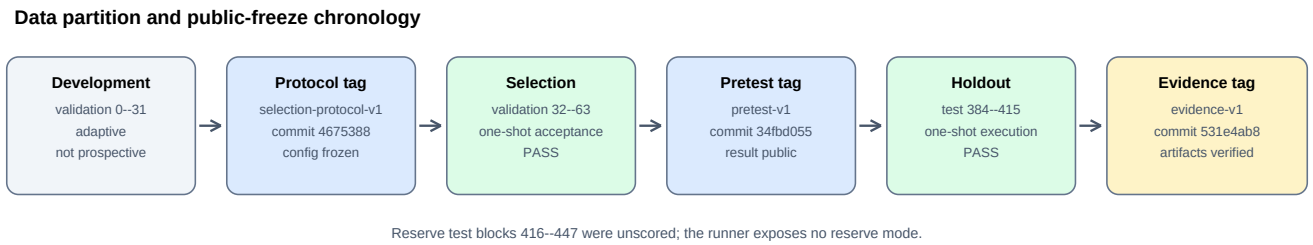

Figure 2: Development, public freezes, and frozen phases. Selection accepted an already fixed configuration; only the later registered test blocks form the prospective holdout result.

Cache NRMSE, cosine similarity, maximum absolute error, and timing are recorded diagnostics but are not acceptance gates.

7 Results

Both frozen phases passed every gate. Selection compressed 150,601,728 canonical bytes to 73,309,770 bytes (2.054320×), with $\Delta\text{NLL} = +0.0005730$ and 4,072/4,096 top-1 agreements. Its block upper bound for ΔNLL was 0.0012222; block and Wilson agreement lower bounds were 99.1762% and 99.1827%, respectively.

On the prospective holdout, 150,601,728 canonical bytes became 73,346,513 complete-container bytes. This is 2.0532909× compression, a reduction of 77,255,215 bytes or 51.2977% relative to the canonical bfloat16 cache. The baseline and decoded-cache NLL values were 2.75956684 and 2.75950591 nat/token, so $\Delta\text{NLL} = -0.00006093$. This small negative difference should be read as compatibility with the fixed tolerance, not evidence that lossy compression improves the model.

Holdout top-1 agreement was 4,071/4,096 (99.3896%), leaving 25 disagreements. The block upper bound for ΔNLL was 0.0005492, the block lower bound for agreement was 99.2472%, and the Wilson lower bound was 99.1543%. The Wilson lower bound was the closest of the three agreement gates to its threshold: 0.1543 percentage points above 99%. Mean KL divergence was 0.00013431 nat. Additional cache diagnostics were NRMSE 0.0032271, cosine similarity 0.99999479, and maximum absolute error 0.406687. These diagnostics do not turn the codec into a lossless method.

Figure 3 exposes the full block-level variation behind the phase aggregates.

The adaptive development observation was slightly stronger on top-1 agreement, but it is not prospective evidence. Showing it in Table 1 makes the tuning boundary explicit rather than pooling all 96 blocks into a larger-looking sample.

8 Evidence and Reproducibility

Every result contains the 32 block records, aggregate values, gate definitions, environment versions, selected-token digest, configuration digest, registration digest, normative implementation digest, attempt-marker links, and a canonical result digest. The frozen implementation uses JSON Schema validation and independently recomputes aggregate arithmetic before the exclusive result write.

The public verifier checks schemas, canonical serialization, physical file hashes, marker/result links, phase order, fixed source ranges, configuration binding, per-block byte accounting, aggregate metrics, Student's t and Wilson bounds, and verdicts. In a tagged clone,

```
python3 \
  RealLLM/verify_voidtoken_v5_evidence.py \
  --require-git-provenance
```

also checks the expected commits and local tag targets. Public remote state and chronology must be corroborated separately. This verifier validates stored evidence; it does not rerun model inference and is not an independent external reproduction.

The canonical holdout result starts with `d1c16e88`; the physical `holdout.json` SHA-256 starts with `499c067d`. Both full values appear in Appendix A. The distinction matters because the canonical digest excludes its own digest field, whereas the file digest covers the exact distributed bytes.

9 Limitations

Narrow experimental scope. Evidence covers one Qwen2.5-0.5B revision, one WikiText-2 construction, 32 holdout blocks, a 383-token prefill, 128 teacher-forced predictions per block, and one Apple-Silicon/MPS environment. It does not establish behavior for longer contexts, different models, datasets, prompts, batch sizes, devices, or precision stacks.

Teacher forcing. Errors do not feed sampled token choices back into the sequence. The result does not measure free-running or sampled generation, downstream

Table 1: Registered phase results. Each phase has 32 blocks and 4,096 teacher-forced predictions. Development was adaptive and is shown only for disclosure; it does not contribute to the prospective verdict. Ratios count every serialized container byte.

Phase	Role / blocks	Ratio	ΔNLL	ΔNLL U95	Top-1	Block L95	Wilson L95	Mean KL	Verdict
Development	adaptive; val. 0–31	2.055836	+0.0008045	0.0013785	99.5605%	99.3638%	99.3548%	0.0001369	not prospective
Selection	one-shot; val. 32–63	2.054320	+0.0005730	0.0012222	99.4141%	99.1762%	99.1827%	0.0001367	PASS
Holdout	prospective; test 384–415	2.053291	-0.0000609	0.0005492	99.3896%	99.2472%	99.1543%	0.0001343	PASS

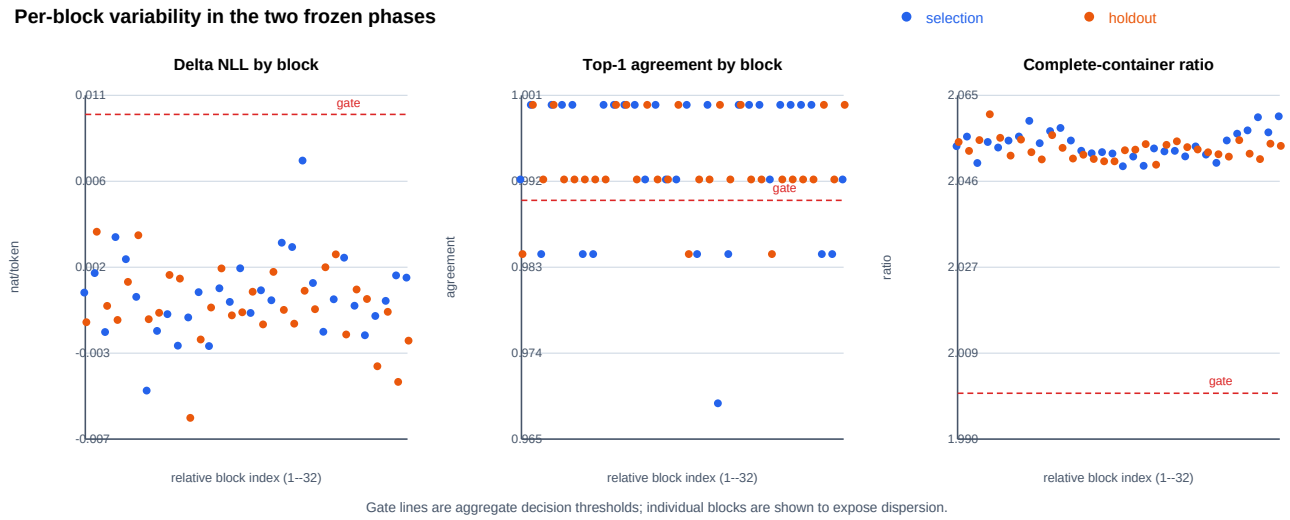

Figure 3: All per-block records: 32 from selection and 32 from holdout. Gate lines show the registered aggregate decision thresholds; a block can lie on the unfavorable side while the prespecified aggregate and confidence gates still pass. No blocks were removed.

task quality, or long-horizon error accumulation. Top-1 agreement is reference-path agreement, not accuracy.

No serving claim. Python encode, decode, and continuation timings are recorded but are not gates and are not optimized-kernel benchmarks. We make no latency, throughput, energy, peak-memory, production-serving, or economic claim.

No comparative baseline. The study does not rerun KIVI, KVQuant, GEAR, QuaRot, KVLinC, SAW-INT4, or KVarN on identical windows. An 8/9-bit codec with approximately 2.05× complete-container compression should not be read as competitive with reported low-bit systems. The contribution is the frozen evidence procedure and bounded artifact result.

Statistical dependence. Token decisions are correlated, blocks are contiguous corpus windows, and the Student-t calculation has only 32 block observations. Registered confidence gates summarize this experiment; they do not prove a universal quality bound.

Process provenance. Durable markers, clean commits, source digests, and public tags constrain the recorded workflow but cannot prove the absence of an unreported private run or human deletion. The earlier v1 test access and full-parquet tokenization are disclosed. External reproduction remains necessary.

Implementation coupling. The fresh strict container parser shares the canonical payload decoder with the encoder self-check. Hash and round-trip coverage catches many faults, but a second independent decoder would provide stronger evidence.

10 Conclusion

VoidToken v5 passed a prospectively frozen, one-shot holdout for complete serialized prefill KV-cache containers on a pinned real language model. The registered result is 2.05329× compression with small NLL difference, 99.3896% top-1 agreement, and all seven aggregate, confidence, and structural gates satisfied. The more durable contribution is a compact evidence chain: adaptive work is separated from prospective phases; byte accounting includes the wire format; code and data choices are pinned; failures and incomplete attempts are terminal; and every published number is traceable to machine-readable records. Broader claims require multi-model,

long-context, free-running, systems-level, and independently reproduced evaluations.

Artifact Availability

Code, frozen records, verifiers, paper source, and deterministic archive builder are available at `https://github.com/ALLPROTO/core-lm-benchmark`. The scientific evidence state is identified by lightweight tag `voidtoken-v5-evidence-v1`; the publication package is identified by `voidtoken-v5-paper-v2`. Repository software is distributed under the MIT License. The arXiv manuscript distribution license is selected separately by the author during submission.

References

[1] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. *arXiv preprint arXiv:2404.00456*, 2024.

[2] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *Advances in Neural Information Processing Systems*, 2024. arXiv:2401.18079.

[3] Jinda Jia, Jisen Li, Zhongzhu Zhou, Jung Hwan Heo, Jue Wang, Tri Dao, Shuaiwen Leon Song, Ben Athiwaratkun, Chenfeng Xu, Tianyi Zhang, and Xiaoxia Wu. SAW-INT4: System-aware 4-bit KV-cache quantization for real-world LLM serving. *arXiv preprint arXiv:2604.19157*, 2026.

[4] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM. *arXiv preprint arXiv:2403.05527*, 2024.

[5] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *Proceedings of the 41st International Conference on Machine Learning*, 2024. arXiv:2402.02750.

[6] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[7] Lorenz K. Muller, Philippe Bich, Chiara Boretti, Hyun-Min Chang, Jiawei Zhuang, and Lukas Cavigelli. KVarN: Variance-normalized KV-cache quantization mitigates error accumulation in reasoning tasks. *arXiv preprint arXiv:2606.03458*, 2026.

[8] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[9] Utkarsh Saxena and Kaushik Roy. KVLinC: KV cache quantization with hadamard rotation and linear correction. *arXiv preprint arXiv:2510.05373*, 2025.

[10] Student. The probable error of a mean. *Biometrika*, 6(1):1–25, 1908.

[11] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

A Evidence Digests

Suite identifier:

`qwen2.5-0.5b-kv-voidtoken-v5-prospective-v1`

Freeze tags:

`voidtoken-v5-selection-protocol-v1`
`voidtoken-v5-pretest-v1`
`voidtoken-v5-evidence-v1`

Final evidence commit:

`531e4ab8d1de61ce93e83164d13caff7bb0759bc`

Protocol-freeze commit:

`467538875402265b2ca915768376e2a5548f3069`

Pretest-freeze commit:

`34fbd0556bd4e8fb889e628ae35175ff596818af`

Model revision:

`060db6499f32faf8b98477b0a26969ef7d8b9987`

Corpus revision:

`b08601e04326c79dfdd32d625aee71d232d685c3`

Configuration, canonical SHA-256:

`4c7be8c836aa725722b51f66dce78af7a5094e887432e622b5322f7ca2cf0af8`

Registration, canonical SHA-256:

`ad5b75791f5385740bcdd472bf81ec546fa17fa59a0715a343ed91a193c1af32`

Normative implementation SHA-256:

`55b2f589aae027e4353cf8011547d821a7d885d1fde8b9c3d9dd7af3cd90d783`

Selection result, canonical SHA-256:

`11329a941051073bae9e2aec3f483f5fc6acf7449ed18457d020f4693c1b1876`

Holdout result, canonical SHA-256:

`d1c16e88655c1fbc9884324742dee3f0b9b4bc86d973c2bf38df3a02cc090eaa`

Holdout result file SHA-256:

`499c067d6ccff4bf1ac4a9f98436a52fa6c414ccced495719532347b89b46167`

Model weights SHA-256:

`88c142557820ccad55bb59756bfcfcf8`

91de9cc6202816bd346445188a0ed342

Both frozen phases recorded macOS 26.3 on arm64, MPS, Python 3.12.13, PyTorch 2.13.0, Transformers 5.14.1, NumPy 2.5.1, tokenizers 0.22.2, safetensors 0.8.0, pyarrow 23.0.1, and zlib 1.2.12. The model ran in float32; only the reference cache was canonicalized through bfloat16.

B Artifact Map

The machine-readable registration is `RealLLM/v5_registration.json`. Normative prose is `RealLLM/V5_PROTOCOL.md`. Adaptive records and their manifest are under `real-llm-v5-development/`; frozen attempt and result artifacts are under `real-llm-v5-results/`. The implementation and frozen runner are `RealLLM/voidtoken_v5.py` and `RealLLM/run_voidtoken_v5_frozen.py`. JSON Schemas, unit tests, an evidence verifier with offline and Git-provenance modes, a deterministic archive builder, and an exact SHA-256 manifest are distributed in the same repository.

The evidence archive intentionally omits the 988 MB model weights and corpus parquet files. Their repository revisions, byte lengths, and hashes are registered so a reproducer can acquire and verify them separately. Exact cross-device logits are not promised; a rerun must report its runtime and compare scientific metrics rather than claim byte-identical MPS output.